Bridge Day 5 STUDENT WORKBOOK

Boolean Combinations and Week 1 Synthesis

Learning sequence: Predict - Trace - Explain - Test - Interpret

Guided engineering evidence workbook

Name		UIN		Section	

Concrete engineering situation

The pump-flow team now has three kinds of evidence from this week: current stored values from Day 2, typed and converted values from Day 3, and boundary comparisons from Day 4. Today the team combines separate evidence checks into a simple go/no-go readiness expression.

The problem today is Boolean combination, not branch routing. You will predict whether combined conditions are `True` or `False`, explain which part controls the result, and name whether your support need is state, type, boundary, or Boolean reasoning.

Engineering task	Decide whether a pump record is ready for supervisor review using separate evidence checks that must all be traceable.
Computational move	Combine condition results with `and`, `or`, and `not` while preserving the reason each condition is true or false.
Python focus	Boolean values, comparison results, `and`, `or`, `not`, parentheses, and compound expression explanation.
Evidence to keep	typed value, boundary result, label check, combined truth value, controlling condition, and support category.

Day 5 Boolean rules

1. `A and B` is `True` only when both parts are `True`.
2. `A or B` is `True` when at least one part is `True`.
3. `not A` reverses a Boolean result.
4. Parentheses make the intended grouping visible.
5. A reliable go/no-go note names the separate evidence checks before naming the combined result.

1. Read separate evidence before combining it

recognize predict

Use the week-one pump record below. Each named condition stores one separate evidence result.

```
# Typed values and boundary limits
flow_lpm = 20.1
low_limit_lpm = 19.6
high_limit_lpm = 20.4
approval_label = "reviewed"
operator_initials = "AM"
retry_count = 0
sensor_ready = True

# Separate evidence checks
lower_ok = flow_lpm >= low_limit_lpm
upper_ok = flow_lpm <= high_limit_lpm
flow_ok = lower_ok and upper_ok
approval_ok = approval_label == "reviewed"
setup_ok = sensor_ready and operator_initials != ""
no_retry_needed = retry_count == 0
record_ready = flow_ok and approval_ok and setup_ok and no_retry_needed
```

Pts.	Prompt	My evidence
1	Which two comparisons create the separate lower and upper boundary checks?	
1	Which condition checks the text label rather than numeric flow?	
1	Which condition checks whether a retry is needed?	
1	Which final expression combines all readiness evidence?	

2. Trace an `and` combination from separate evidence

trace explain

Complete the table. Do not skip directly to the final result; show each part.

Pts.	Combined condition	Left part	Right part	Combined result and reason
2	`lower_ok and upper_ok`			
2	`sensor_ready and operator_initials != ""`			
2	`flow_ok and approval_ok`			
2	`setup_ok and no_retry_needed`			

State/type/boundary connection

A Boolean expression is only as trustworthy as the evidence pieces inside it. If a part came from a string label, a converted number, or a boundary comparison, keep that source visible in your reason.

3. Use `or` and `not` for follow-up evidence

predict explain

A record may need follow-up if at least one risk is present. Complete the table and explain which part controls the result.

```
needs_flow_review = not flow_ok
needs_approval_review = not approval_ok
needs_setup_review = not setup_ok
needs_followup = needs_flow_review or needs_approval_review or needs_setup_review
```

Pts.	Expression	True or False?	Reason
2	`not flow_ok`		
2	`not approval_ok`		
2	`needs_flow_review or needs_approval_review`		
2	`needs_followup`		

Common trap to check

`or` does not mean every risk is present. It means at least one part is `True`. A good note says which part made the expression true.

4. Build a Week 1 condition table

trace transfer

For each case, use Day 3 type/label evidence and Day 4 boundary evidence before deciding the combined result.

Pts.	Flow	Approval label	`flow_ok`	`approval_ok`	`setup_ok`	`record_ready` and reason
2	`20.1`	`"reviewed"`				
2	`20.5`	`"reviewed"`				
2	`20.0`	`"pending"`				
2	`19.6`	`"reviewed"` with blank initials				

Bridge to Week 2

Week 2 will use these Boolean results to choose branch outcomes with `if`/`else`. Day 5 stops one step earlier: decide the truth value and explain the evidence, but do not write a branch yet.

5. Debug a Boolean readiness claim

debug test explain

A teammate writes this note after seeing that `approval_label` stores a non-empty string:

Readiness note to debug

The record is ready because the flow is in range or the approval label has text.

Pts.	Prompt	My response
2	What Day 3 mistake appears in the teammate's note?	
2	What Day 4 boundary evidence is still needed before trusting `flow_ok`?	
2	Why is `or` unsafe if the engineering rule requires both flow evidence and approval evidence?	
2	Rewrite the readiness note so it names the separate evidence checks and uses `and` correctly.	

Week 1 synthesis habit

When a combined expression surprises you, diagnose the support need by source: current state from Day 2, type/cast/truthiness from Day 3, boundary comparison from Day 4, or Boolean connector from Day 5.

6. Mastery check and support next step

transfer explain

Use your own evidence from this workbooklet. Do not write a generic answer.

Pts.	Prompt	My response
2	Give one example where an `and` expression is false because one required evidence check failed.	
2	In one or two sentences, explain how Day 2 state, Day 3 type evidence, and Day 4 boundary evidence work together in Day 5.	

My mastery check

Mark the strongest statement that is true after this workbooklet.

☐	Secure: I can combine separate evidence checks with `and`, `or`, and `not`, and explain which part controls the result.
☐	Developing: I can evaluate some combined conditions, but I still need support identifying which evidence check failed.
☐	Needs support: I need a smaller example before I can combine state, type, boundary, and Boolean evidence.

My next support move

My issue is mostly: setup / Python syntax / tracing / state history / type conversion / boundary cases / Boolean connectors / confidence / time.
The first evidence source where I got uncertain was: _____
My next small action is: ask a partner / ask instructor / rebuild the condition table / test one failed condition at a time / try the recovery version.

Workbooklet target: I can combine typed values, boundary checks, approval evidence, and Boolean connectors into a traceable Week 1 go/no-go explanation.

Bridge Day 5 STUDENT WORKBOOK

VIP Evidence Handoff

Learning sequence: Predict - Trace - Explain - Test - Interpret

Contract 07a04826c975 • longitudinal template 07242026_v1

Why engineers use this page

Combine separate comparison evidence with visually distinct Boolean operators, then complete the notebook's independent decision programs.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: comparison expression, boolean operator, if elif else, abs call, counter update
Carried authoring tools: assignment, print call, numeric literal, arithmetic expression, input call, int conversion, float conversion, f string
Supplied-code exposure only: for loop, list traversal
An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.13	Compare With Tolerance	`18-13-work / 18-13-transfer / 18-13-cold`	comparison expression, f string, float conversion, input call
18.14	Count High Readings	`18-14-work / 18-14-transfer / 18-14-cold`	arithmetic expression, comparison expression, counter update, f string, if elif else
18.15	Lamination Fee	`18-15-work / 18-15-transfer / 18-15-cold`	comparison expression, f string, if elif else, input call, int conversion
18.16	Temperature Status	`18-16-work / 18-16-transfer / 18-16-cold`	comparison expression, f string, float conversion, if elif else, input call
18.17	Pickup Window	`18-17-work / 18-17-transfer / 18-17-cold`	boolean operator, comparison expression, f string, if elif else, input call, int conversion

Readiness evidence

1. For one mapped activity, write each component Boolean on its own line before combining anything with 'and', 'or', or 'not'.

2. Explain which case must be rejected first in Activity 18.17 and why that priority prevents an unsafe quick-pickup result.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	
Observed Result	
Revision Reason	
Engineering Meaning	
Units Or Not Applicable	
Assumption	
Model Limit	
Next Test	
Help Trigger	
Minutes Used	

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Generated from the canonical VIP construct and evidence contract; workbook planning and executable notebook work remain distinct.